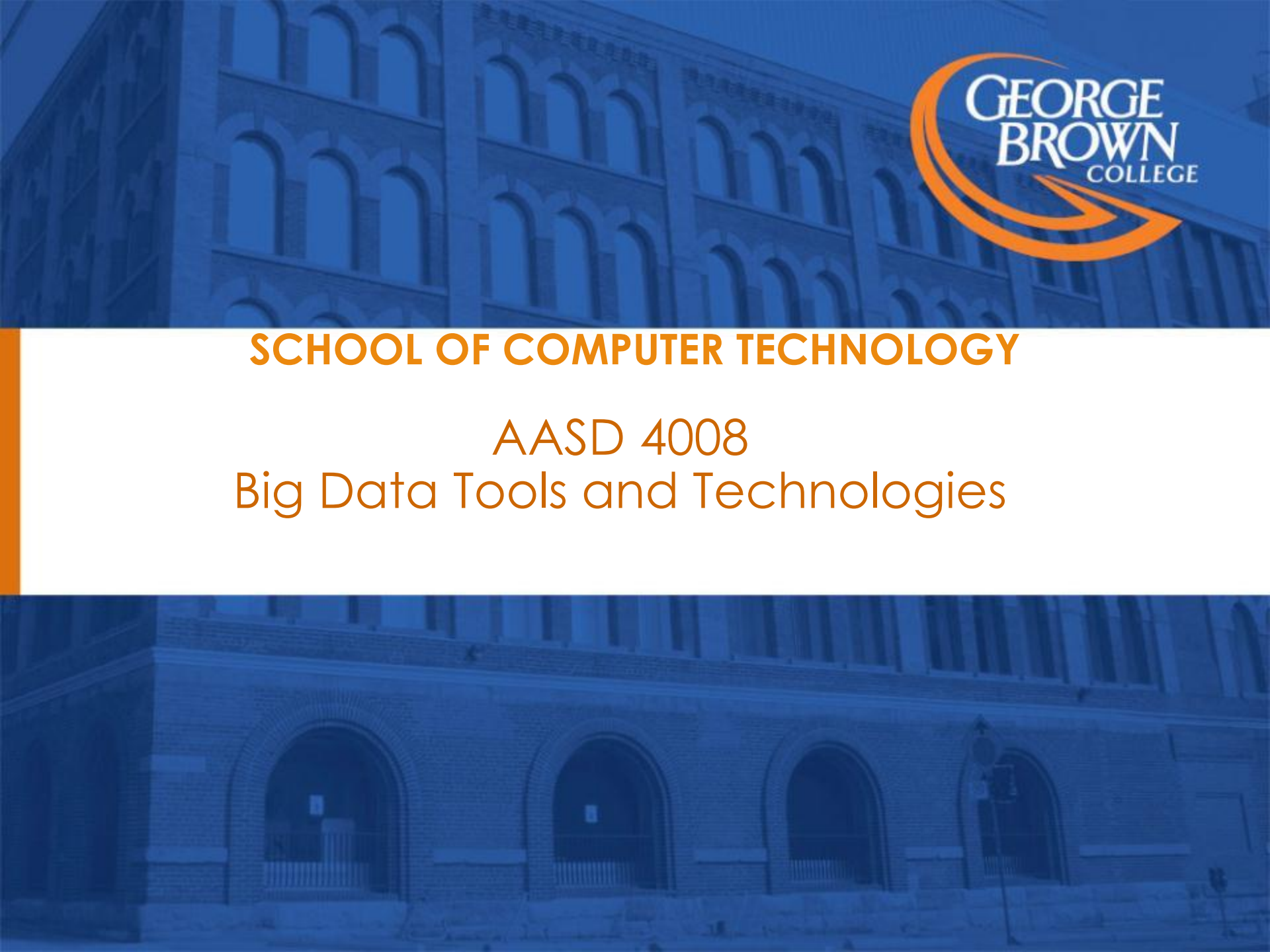


SCHOOL OF COMPUTER TECHNOLOGY

AASD 4008
Big Data Tools and Technologies

Topic 4 – Processing semi–Structured Data Mongodb

We discussed what semi-structured data in the previous section is; now, in this section, we will further explore the container of a semi structure data, MongoDB!

Topic 4 – Processing semi–Structured Data Mongodb

Introduction

MongoDB is a popular document-oriented NoSQL database that provides high scalability and flexibility for storing and retrieving data. It uses a JSON-like format called BSON (Binary JSON) to represent data, making it easy to work with. MongoDB supports automatic sharding for horizontal scaling and offers robust features like indexing, replication, and aggregation pipelines for efficient data manipulation. Its flexible schema allows for dynamic updates and easy handling of unstructured data. MongoDB is widely used in modern web and mobile applications due to its ability to handle large volumes of data and provide fast, real-time data access.

MongoDB is the most popular document-based database used by millions of de and adopted by many top-ranking organizations.

1 Docs.Mongodb.com : https://docs.mongodb.com/

2 Mongodb university : https://university.mongodb.com/

Some concepts

Cluster	Group of MongoDB servers that store your data. Serves are configured in form of a replica Set
Replica Set	A Replica Set is few connected machines that store the same data to ensure that if something happens to one of the machines, the data will remain intact.
Instance	Instance is a single machine locally or in the cloud, running a MongoDB database
Sharded cluster	Sharding distributes data across the shards in the cluster, allowing each shard to contain a subset of the total cluster data. As the data set grows, additional shards increase storage capacity of the cluster
Database	Database is a physical container of collections
Collection – AS GOOD AS SQL TABLE	A collection is an organized store of documents in MongoDB usually with common fields between documents. There can be many collections per database and many documents per collections
Document AS GOOD AS SQL RECORD	A Document is a way to organize and store data as set of field-value pairs. Documents have dynamic schema means that, 1. Document in the same collection don't need to have the same set of fields or structure. 2. Common field in the collection's document may hold different type of data.
Field	A field is a unique identifier for a data point
Value	A value is the data related to a given identifier

1

2

1 Docs.Mongodb.com : https://docs.mongodb.com/

2 Mongodb university : https://university.mongodb.com/

JSON

MongoDB stores data in **BSON** and to understand BSON, you need to first understand **JSON**

JSON is Java Script object notation

1. Start and end the document with {}
2. Separate Each Key and Value with a colon :
3. Separate each Key and Value pair with a Coma ,
4. Include “” around keys “keys”

Pros → User-friendly , readable ,familiar and Human/Machine readable

Cons → text based (slow), space consuming and limited data types

[1] [2]

```
{
  "_id" : "10021-2015-ENFO",
  "certificate_number" : 9278806,
  "business_name" : "ATLIXCO DELI",
  "date" : "Feb 20 2015",
  "result" : "No Violation Issued",
  "sector" : "Cigarette Retail - 127",
  "address" : {
      "city" : "RIDGEWOOD",
      "zip" : 11385,
      "street" : "MENAHAN ST",
      "number" : 1712
    }
}
```


1 Docs.Mongodb.com : https://docs.mongodb.com/

2 Mongodb university : https://university.mongodb.com/

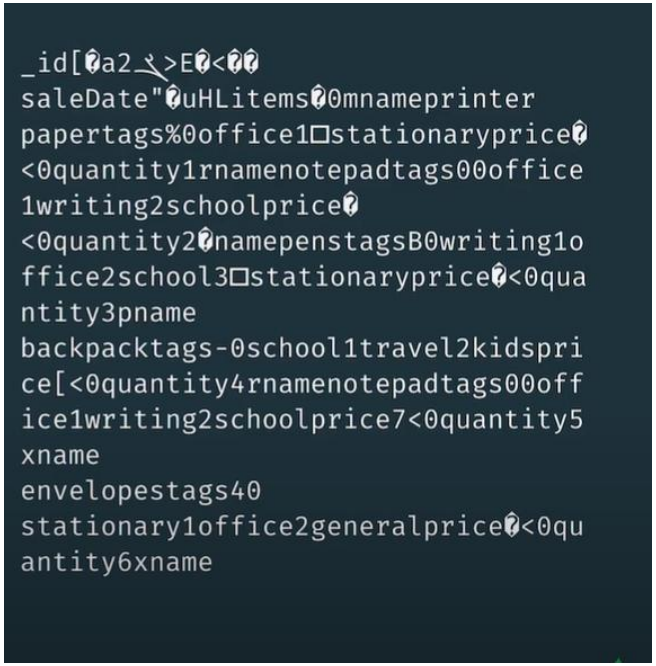
BSON

With the limited capacity and drawback of JSON, MongoDB **stores and transfer** data between networks using BSON (**Binary** JSON)

Pros -> speed, space, and flexibility (new data types, date and raw binary).

Cons → only readable by machine

Mongodb display data in **JSON**



1 Docs.Mongodb.com : https://docs.mongodb.com/

2 Mongodb university : https://university.mongodb.com/

NoSQL vs SQL Database

NoSQL ("non-SQL" or "not only SQL") databases were developed in the late 2000s with a focus on scaling, fast queries, allowing for frequent application changes, and making programming more straightforward for developers. [1] [2]

Relational databases accessed with SQL (Structured Query Language) were developed in the 1970s, focusing on reducing data duplication, as storage was much more costly than developer time. SQL databases have rigid, complex, tabular schemas and typically require expensive vertical scaling.

Feature	SQL Database	NoSQL Database
Data Storage Model	Tables with fixed rows and columns	Document: JSON documents, Key-value: key-value pairs, Wide-column: tables with rows and dynamic columns
Development History	Developed in the 1970s with a focus on reducing data duplication	Developed in the late 2000s with a focus on scaling and allowing. for rapid application change driven by agile and DevOps practices.
Examples	Oracle, MySQL, Microsoft SQL Server, and PostgreSQL	Document: MongoDB CosmosDB and CouchDB, Key-value: Redis and DynamoDB, Wide-column: Cassandra and Hbase

1 Docs.Mongodb.com : https://docs.mongodb.com/

2 Mongodb university : https://university.mongodb.com/

NoSQL vs SQL Database- Continued !

Feature	SQL Database	NoSQL Database
Primary Purpose	General purpose	Document: general purpose, Key-value: large amounts of data with simple lookup queries, Wide-column: large amounts of data with predictable query patterns
Schemas	Rigid	Flexible
Multi-Record ACID Transactions	Supported Follows ACID principle, Atomicity, Consistency, Isolation, and Durability. All connections have same results with consistency.	MongoDB is an ACID Complaint database as of Version 4.2. provides **Eventually consistency.**

2 1

1 Docs.Mongodb.com : https://docs.mongodb.com/

2 Mongodb university : https://university.mongodb.com/

NoSQL vs SQL Database – Continued !

Feature	SQL Database	NoSQL Database
Scaling	Vertical (scale-up with a larger server) RDBMS is vertically scalable. Performance increases with increase of RAM. Vertical scaling can essentially resize the server with no change to the code by adding more resources (Space and RAM!!) ***Example of apartment building or restaurant***	Horizontal (scale-out across commodity servers) MongoDB is horizontally scalable as well. Its performance increases with addition of processor. Horizontal scaling affords the ability to scale wider to deal with traffic. Its ability to connect multiple hardware or software such as servers so that they work as a single logical unit ***Example of highway***
Joins	Typically required	Typically, not required
Data to Object Mapping	Requires ORM (object-relational mapping)	Many do not require ORMs. MongoDB documents map directly to data structures in most popular programming languages.

Terminologies

RDBMS	MongoDB
Database	Database
Tables	Collection
Row	Document
Columns	Field
Table Join	Embedded Documents – also lookups
Primary Key	PrimaryKey (default _id provided by MongoDB)
Normalized	DE Normalized

1 Docs.Mongodb.com : https://docs.mongodb.com/

2 Mongodb university : https://university.mongodb.com/

CAP Theorem
Consistency–Availability–Partition

The CAP theorem is a fundamental concept in distributed systems that states that it is **impossible** to achieve all three properties of Consistency (C), Availability (A), and Partition Tolerance (P) simultaneously in a distributed system.

Consistency (C): Consistency refers to the requirement that all nodes in a distributed system have **the same view of the data at all times**. Whenever data is updated, all subsequent reads should return the updated value. Strong consistency ensures immediate visibility of changes across the system.

Availability (A): Availability means that every request to the distributed system receives a response, even in the presence of failures or network partitions. The system remains operational and accessible to clients, providing a timely response to read and write requests.

Partition Tolerance (P): Partition tolerance addresses the ability of the system to continue functioning despite network partitions or communication failures that may occur in a distributed environment. Network partitions can cause subsets of nodes to be isolated from each other, leading to potential inconsistencies.

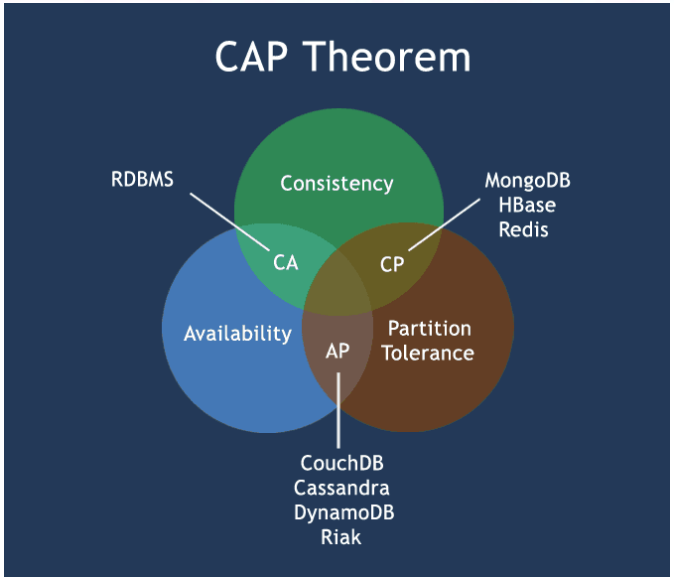
CAP Theorem
Consistency–Availability–Partition

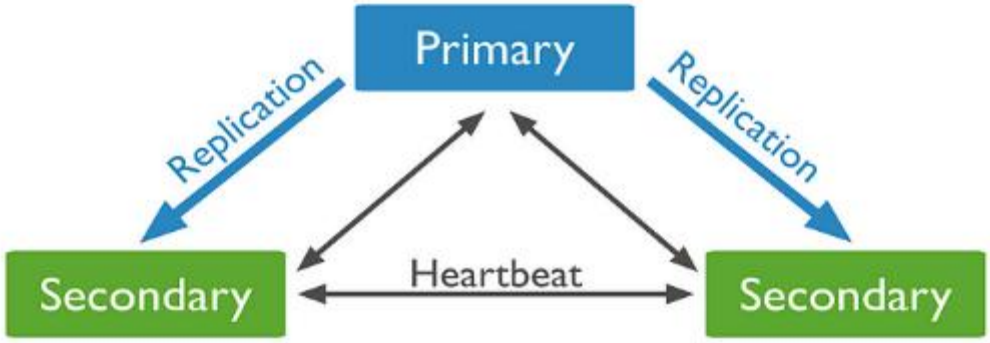
According to the CAP theorem, when a distributed system faces a network partition (P), it must make a trade-off between consistency (C) and availability (A). This means that during a partition, a system can either prioritize maintaining consistency and sacrifice availability **(CP),** or prioritize availability and sacrifice strict consistency **(AP).**

In practical terms, this means that in a distributed system, you have to choose whether you want to focus on providing immediate consistency even during network partitions (CP), or prioritize availability by accepting potential inconsistencies until the system can reconcile the data (AP).

MongoDB's default behavior is CP

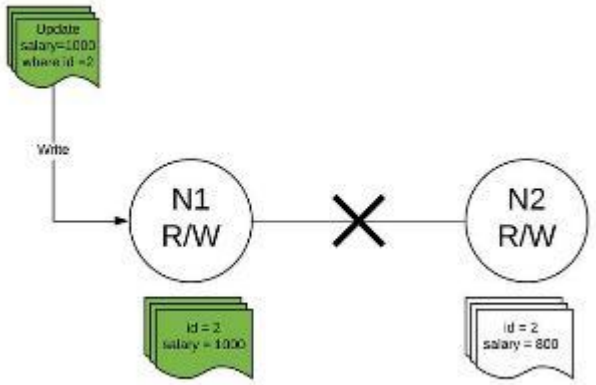
CAP Theorem
medium.com





CAP Theorem-Mongodb- Example medium.com

Scenario 1: Failing to propagate update request to other nodes.
Say, we have two nodes(N1 & N2) in a cluster and both nodes can accept read and write requests.

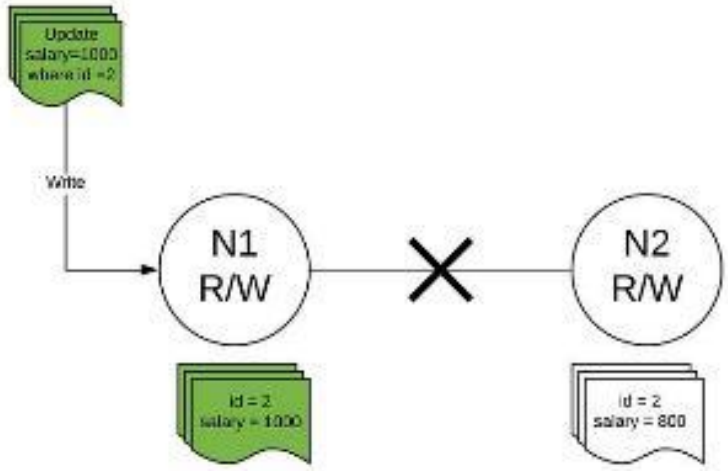


CAP Theorem-Mongodb- Example medium.com

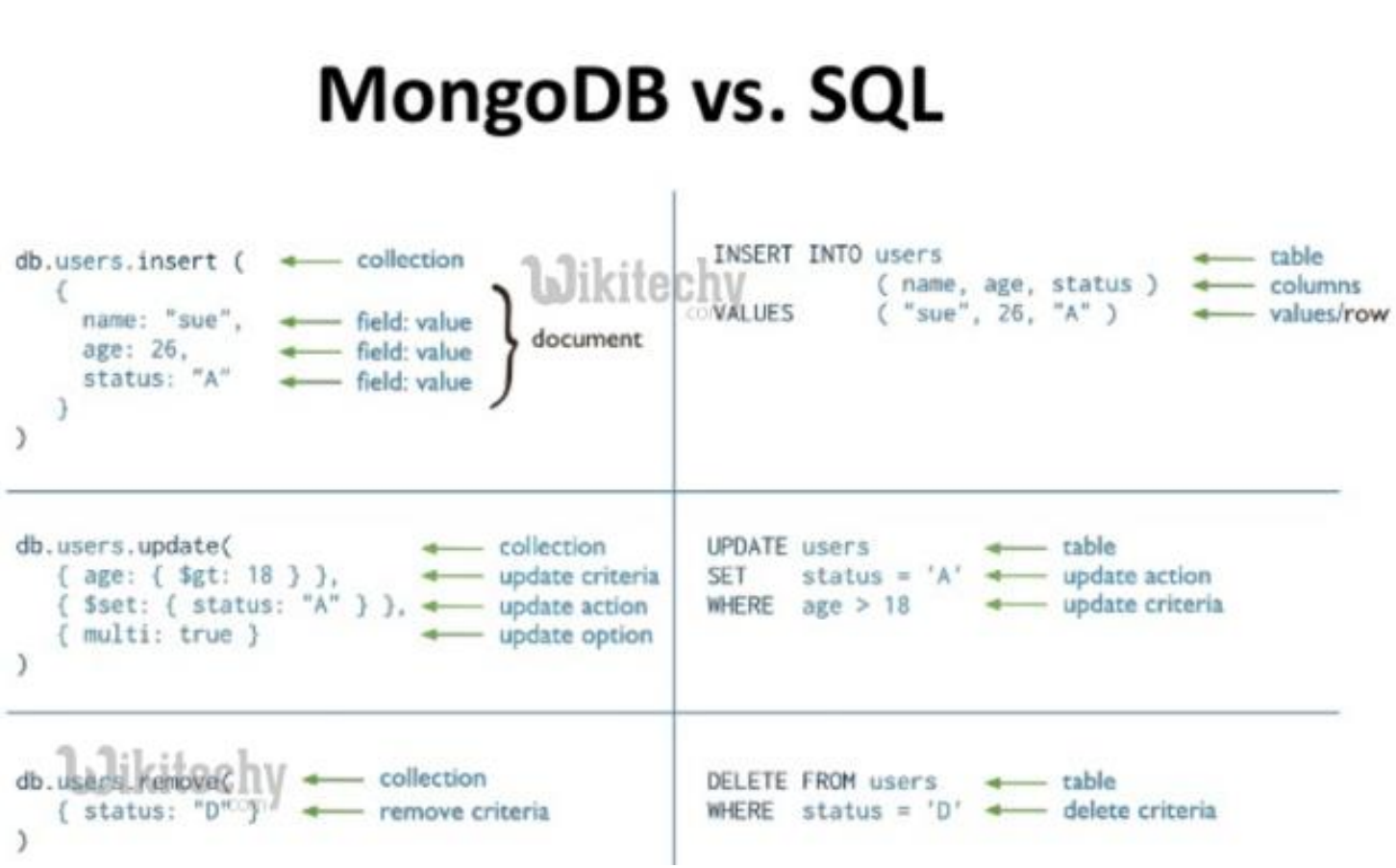
In the diagram, the N1 node gets an update request for **id 2** and updates the salary from 800 to 1000. But, since there is network partition, hence, N1 can not send the latest update to N2.

So, when a read request comes to N2, it can do either of two things:

1. Respond with whatever data it has, in this case, salary = 800, and update the data when the network partition resolves — **making the system Available but Inconsistent.**
2. Simply return with an error, saying "I do not have updated data" — **making a system Consistent but Unavailable** by not returning inconsistent data.

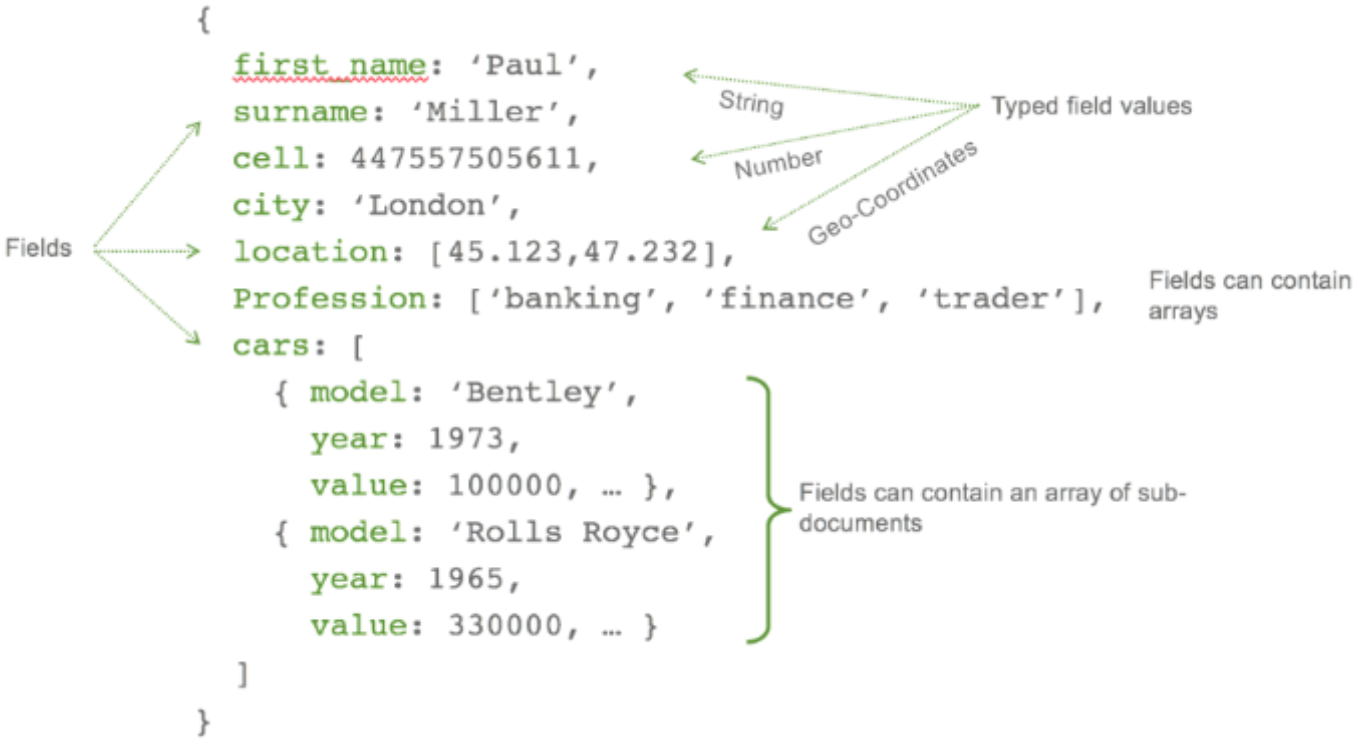


MongoDB vs. SQL



[1] Wikitechy.com : https://www.wikitechy.com/tutorials/node-js/node-js-mongodb-insert-record

Anatomy of a Sample Document (record)



1. Docs.Mongodb.com : https://docs.mongodb.com/
2. Mongodb university : https://university.mongodb.com/

Benefits of NoSQL Database

GEORGE BROWN COLLEGE

1 2

Flexible data models

NoSQL databases typically have very flexible schemas. A flexible schema allows you to make changes to your database as requirements change easily. You can iterate quickly and continuously integrate new application features to provide value to your users faster.

The structure is clear! no complex joins.

Horizontal scaling

Most SQL databases require you to scale up vertically (migrate to a larger, more expensive server) when you exceed the capacity requirements of your current server. Conversely, most NoSQL databases allow you to scale horizontally, meaning you can add cheaper commodity servers whenever you need to.

1 Docs.Mongodb.com : https://docs.mongodb.com/

2 Mongodb university : https://university.mongodb.com/

MongoDB – Data Modeling Introduction

The key challenge in data modeling is balancing the needs of the application, the performance characteristics of the database engine, and the data retrieval patterns. When designing data models, always consider the application usage of the data (i.e. queries, updates, and processing of the data) as well as the inherent structure of the data itself.

Flexible Schema

Unlike SQL databases, where you must determine and declare a table's schema before inserting data, MongoDB's collections, by default, do not require their documents to have the same schema. That is:

- The documents in a single collection do not need to have the same set of fields and the data type for a field can differ across documents within a collection.

- To change the structure of the documents in a collection, such as add new fields, remove existing fields, or change the field values to a new type, update the documents to the new structure.

https://docs.mongodb.com/upcoming/core/data-modeling-introduction/

1 Docs.Mongodb.com : https://docs.mongodb.com/

2 Mongodb university : https://university.mongodb.com/

MongoDB Data Modeling Design

1 2

Effective data models support our application needs. The key consideration for the structure of your document is the decision to **embed** or use **reference** *(normalized) . For further reading !*

- *https://docs.mongodb.com/upcoming/core/data-model-design/*
- *https://docs.mongodb.com/upcoming/core/data-modeling-introduction/*

(a) Embedded Data Model

```
{
  _id: <ObjectId1>,
  username: "123xyz",
  contact: {
             phone: "123-456-7890",
             email: "xyz@example.com"
           },
  access: {
            level: 5,
            group: "dev"
          }
}
```

Embedded sub-document

Embedded sub-document

(b) Normalized Data Model

user document

```
{
  _id: <ObjectId1>,
  username: "123xyz"
}
```

contact document

```
{
  _id: <ObjectId2>,
  user_id: <ObjectId1>,
  phone: "123-456-7890",
  email: "xyz@example.com"
}
```

access document

```
{
  _id: <ObjectId3>,
  user_id: <ObjectId1>,
  level: 5,
  group: "dev"
}
```

3

1 Docs.Mongodb.com : https://docs.mongodb.com/

2 Mongodb university : https://university.mongodb.com/

3 Devopedia.org : https://devopedia.org/data-modelling-with-mongodb

MongoDB Data Example and Patterns – Relationships

Now let's discuss relationship between documents. There are 3 main categories

1. One to One Relationship
    1. Embedded
    2. Subset pattern
2. One to many Relationship
    1. Embedded
    2. Subset pattern
3. Many to Many Relationship with references

1 Docs.Mongodb.com : https://docs.mongodb.com/

2 Mongodb university : https://university.mongodb.com/

One to One Relation

This model uses embedded documents to describe a one-to-one relationship between connected data. Embedding connected data in a **single document can reduce the number of read operations** required to get data. You should structure your schema, so your application receives all its required information in a single read operation.

https://docs.mongodb.com/upcoming/tutorial/model-embedded-one-to-one-relationships-between-documents/

1 Docs.Mongodb.com : https://docs.mongodb.com/

2 Mongodb university : https://university.mongodb.com/

One to One Relation - Embedded

Rule of thumb. If the data is queries together, it should be kept together. Consider the 2 documents Patron and address, a patron can have one address (or more!)

```
// patron document
{
   _id: "joe",
   name: "Joe Bookreader"
}

// address document
{
   patron_id: "joe", // reference to patron document
   street: "123 Fake Street",
   city: "Faketon",
   state: "MA",
   zip: "12345"
}
```

If the address data frequently retrieved with name, then a better model would embed the address data into pattern data.

```
{
   _id: "joe",
   name: "Joe Bookreader",
   address: {
              street: "123 Fake Street",
              city: "Faketon",
              state: "MA",
              zip: "12345"
            }
}
```


1 Docs.Mongodb.com : https://docs.mongodb.com/

2 Mongodb university : https://university.mongodb.com/

One to One Relation – Subset Pattern

A potential problem with the embedded document pattern is that it can lead to large documents **that contain fields that the application does not need. This unnecessary data can cause extra load on your server** and slow down read operations. Instead, you can use the subset pattern to retrieve the subset of data which is accessed the most frequently in a single database call.

Consider an application that shows information on movies. The database contains a movie collection with the following schema:


```
{
  "_id": 1,
  "title": "The Arrival of a Train",
  "year": 1896,
  "runtime": 1,
  "released": ISODate("01-25-1896"),
  "poster": "http://ia.media-imdb.com/images/M/MV5BMjEyNDk5MDYzOV5BMl5BanBnXkFtZTgwNjIxMTEwM
  "plot": "A group of people are standing in a straight line along the platform of a railway
  "fullplot": "A group of people are standing in a straight line along the platform of a rai
  "lastupdated": ISODate("2015-08-15T10:06:53"),
  "type": "movie",
  "directors": [ "Auguste Lumière", "Louis Lumière" ],
  "imdb": {
    "rating": 7.3,
    "votes": 5043,
    "id": 12
  },
  "countries": [ "France" ],
  "genres": [ "Documentary", "Short" ],
  "tomatoes": {
    "viewer": {
      "rating": 3.7,
      "numReviews": 59
    },
    "lastUpdated": ISODate("2020-01-09T00:02:53")
  }
}
```


1 Docs.Mongodb.com : https://docs.mongodb.com/

2 Mongodb university : https://university.mongodb.com/

One to One Relation – Subset Pattern - Continued

Let us assume that application doesn't need to read Full plot, poster and plot data frequently together, in this scenario, we can split the document into 2, one for movie collection, second for movie detail.

```
// movie collection

{
  "_id": 1,
  "title": "The Arrival of a Train",
  "year": 1896,
  "runtime": 1,
  "released": ISODate("1896-01-25"),
  "type": "movie",
  "directors": [ "Auguste Lumière", "Louis Lumière" ],
  "countries": [ "France" ],
  "genres": [ "Documentary", "Short" ],
}
```

```
// movie_details collection

{
  "_id": 156,
  "movie_id": 1, // reference to the movie collection
  "poster": "http://ia.media-imdb.com/images/M/MV5BMjEyNDk5MDYzOV5BMl5BanBnXkFtZTgwNjIx
  "plot": "A group of people are standing in a straight line along the platform of a ra
  "fullplot": "A group of people are standing in a straight line along the platform of
  "lastupdated": ISODate("2015-08-15T10:06:53"),
  "imdb": {
    "rating": 7.3,
    "votes": 5043,
    "id": 12
  },
  "tomatoes": {
    "viewer": {
      "rating": 3.7,
      "numReviews": 59
    },
    "lastUpdated": ISODate("2020-01-29T00:02:53")
  }
}
```

1 Docs.Mongodb.com : https://docs.mongodb.com/

2 Mongodb university : https://university.mongodb.com/

One to Many Relation

[1] [2]

One to many – Embedded is a data model that uses embedded documents to describe a one-to-many relationship between connected data. Embedding connected data in a single document can reduce the number of read operations required to obtain data. In general, you should structure your schema, so your application receives all of its required information in a single read operation.

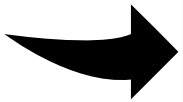
https://docs.mongodb.com/upcoming/tutorial/model-embedded-one-to-many-relationships-between-documents/

1 Docs.Mongodb.com : https://docs.mongodb.com/

2 Mongodb university : https://university.mongodb.com/

One to Many Relation - Embedded

Consider this example that maps patron and multiple address relationships. The example illustrates the advantage of embedding over referencing if you need to view many data entities in context of another. In this one-to-many relationship between **patron** and **address** data, the **patron** has **multiple address** entities. In the normalized data model, the address documents contain a reference to the patron document



```
// patron document
{
   _id: "joe",
   name: "Joe Bookreader"
}

// address documents
{
   patron_id: "joe", // reference to patron document
   street: "123 Fake Street",
   city: "Faketon",
   state: "MA",
   zip: "12345"
}

{
   patron_id: "joe",
   street: "1 Some Other Street",
   city: "Boston",
   state: "MA",
   zip: "12345"
}
```

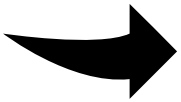

1 Docs.Mongodb.com : https://docs.mongodb.com/

2 Mongodb university : https://university.mongodb.com/

One to Many Relation – Embedded – Continued !

If your application frequently retrieves the address data with the name information, then your application needs to issue multiple queries to resolve the references.

A more optimal schema would be to embed the address data entities in the patron data, as in the following document:



```
{
  "_id": "joe",
  "name": "Joe Bookreader",
  "addresses": [
                {
                  "street": "123 Fake Street",
                  "city": "Faketon",
                  "state": "MA",
                  "zip": "12345"
                },
                {
                  "street": "1 Some Other Street",
                  "city": "Boston",
                  "state": "MA",
                  "zip": "12345"
                }
              ]
}
```

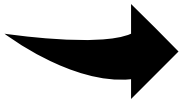
1 Docs.Mongodb.com : https://docs.mongodb.com/

2 Mongodb university : https://university.mongodb.com/

One to Many Relation – Subset Pattern

A potential problem with the embedded document pattern is that it can lead to large documents, especially if the embedded field is unbounded. In this case, you can use the subset pattern to only access data which is required by the application, instead of the entire set of embedded data.

Consider an e-commerce site that has a list of reviews for a product:



```
{
  "_id": 1,
  "name": "Super Widget",
  "description": "This is the most useful item in your toolbox.",
  "price": { "value": NumberDecimal("119.99"), "currency": "USD" },
  "reviews": [
    {
      "review_id": 786,
      "review_author": "Kristina",
      "review_text": "This is indeed an amazing widget.",
      "published_date": ISODate("2019-02-18")
    },
    {
      "review_id": 785,
      "review_author": "Trina",
      "review_text": "Nice product. Slow shipping.",
      "published_date": ISODate("2019-02-17")
    },
    ...
    {
      "review_id": 1,
      "review_author": "Hans",
      "review_text": "Meh, it's okay.",
      "published_date": ISODate("2017-12-06")
    }
  ]
}
```

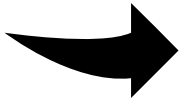

1 Docs.Mongodb.com : https://docs.mongodb.com/

2 Mongodb university : https://university.mongodb.com/

One to Many Relation – Subset Pattern - Continued

The reviews are sorted in reverse chronological order. When a user visits a product page, the application loads the ten most recent reviews.

Instead of storing all of the reviews with the product, you can split the collection into two collections:



```
{
  "_id": 1,
  "name": "Super Widget",
  "description": "This is the most useful item in your toolbox.",
  "price": { "value": NumberDecimal("119.99"), "currency": "USD" },
  "reviews": [
    {
      "review_id": 786,
      "review_author": "Kristina",
      "review_text": "This is indeed an amazing widget.",
      "published_date": ISODate("2019-02-18")
    },
    {
      "review_id": 785,
      "review_author": "Trina",
      "review_text": "Nice product. Slow shipping.",
      "published_date": ISODate("2019-02-17")
    },
    ...
    {
      "review_id": 1,
      "review_author": "Hans",
      "review_text": "Meh, it's okay.",
      "published_date": ISODate("2017-12-06")
    }
  ]
}
```


1 Docs.Mongodb.com : https://docs.mongodb.com/

2 Mongodb university : https://university.mongodb.com/

One to Many Relation – Subset Pattern - Continued

The **Product** collection stores information on each product, including product's **ten** most recent reviews

```
{
  "_id": 1,
  "name": "Super Widget",
  "description": "This is the most useful item in your toolbox.",
  "price": { "value": NumberDecimal("119.99"), "currency": "USD" },
  "reviews": [
    {
      "review_id": 786,
      "review_author": "Kristina",
      "review_text": "This is indeed an amazing widget.",
      "published_date": ISODate("2019-02-18")
    }
    ...
    {
      "review_id": 776,
      "review_author": "Pablo",
      "review_text": "Amazing!",
      "published_date": ISODate("2019-02-16")
    }
  ]
}
```

1 The **Review** collection stores all reviews.

2 Each review contains a reference to the product for which it was written

```
{
  "review_id": 786,
  "product_id": 1,
  "review_author": "Kristina",
  "review_text": "This is indeed an amazing widget.",
  "published_date": ISODate("2019-02-18")
}
{
  "review_id": 785,
  "product_id": 1,
  "review_author": "Trina",
  "review_text": "Nice product. Slow shipping.",
  "published_date": ISODate("2019-02-17")
}
...
{
  "review_id": 1,
  "product_id": 1,
  "review_author": "Hans",
  "review_text": "Meh, it's okay.",
  "published_date": ISODate("2017-12-06")
}
```

By storing the ten most recent reviews in the `product` collection, query will only collect a subset of the overall data and If a user wants to see additional reviews, the application makes a call to the `review` collection.

1 Docs.Mongodb.com : https://docs.mongodb.com/

2 Mongodb university : https://university.mongodb.com/

One to Many Relation – References

[1] [2]

This section describes a data model that uses references between documents to describe one-to-many relationships between connected data.

Consider the following example that maps publisher and book relationships. The example illustrates the advantage of referencing over embedding to avoid repetition of the publisher information.

Embedding the publisher document inside the book document would lead to repetition of the publisher data, as the following documents show:

https://docs.mongodb.com/upcoming/tutorial/model-referenced-one-to-many-relationships-between-documents/

1. Docs.Mongodb.com : https://docs.mongodb.com/
2. Mongodb university : https://university.mongodb.com/

One to Many Relation – References - Continued

To avoid repetition of the **publisher** data, use *references* and keep the publisher information in a separate collection from the book collection.

```
{
    title: "MongoDB: The Definitive Guide",
    author: [ "Kristina Chodorow", "Mike Dirolf" ],
    published_date: ISODate("2010-09-24"),
    pages: 216,
    language: "English",
    publisher: {
                name: "O'Reilly Media",
                founded: 1980,
                location: "CA"
              }
}

{
    title: "50 Tips and Tricks for MongoDB Developer",
    author: "Kristina Chodorow",
    published_date: ISODate("2011-05-06"),
    pages: 68,
    language: "English",
    publisher: {
                name: "O'Reilly Media",
                founded: 1980,
                location: "CA"
              }
}
```

1. Docs.Mongodb.com : https://docs.mongodb.com/
2. Mongodb university : https://university.mongodb.com/

One to Many Relation – References - Continued

When using references, the growth of the relationships determine where to store the reference. If the number of books per publisher is small with limited growth, storing the book reference inside the publisher document may sometimes be useful. Otherwise, if the number of books per publisher is unbounded, this data model would lead to mutable, growing arrays, as in the following example:

```
{
  name: "O'Reilly Media",
  founded: 1980,
  location: "CA",
  books: [123456789, 234567890, ...]
}

{
  _id: 123456789,
  title: "MongoDB: The Definitive Guide",
  author: [ "Kristina Chodorow", "Mike Dirolf" ],
  published_date: ISODate("2010-09-24"),
  pages: 216,
  language: "English"
}

{
  _id: 234567890,
  title: "50 Tips and Tricks for MongoDB Developer",
  author: "Kristina Chodorow",
  published_date: ISODate("2011-05-06"),
  pages: 68,
  language: "English"
}
```

1. Docs.Mongodb.com : https://docs.mongodb.com/
2. Mongodb university : https://university.mongodb.com/

One to Many Relation – References - Continued

To avoid mutable, growing arrays, store the publisher reference inside the book document:

```
{
    _id: "oreilly",
    name: "O'Reilly Media",
    founded: 1980,
    location: "CA"
}

{
    _id: 123456789,
    title: "MongoDB: The Definitive Guide",
    author: [ "Kristina Chodorow", "Mike Dirolf" ],
    published_date: ISODate("2010-09-24"),
    pages: 216,
    language: "English",
    publisher_id: "oreilly"
}

{
    _id: 234567890,
    title: "50 Tips and Tricks for MongoDB Developer",
    author: "Kristina Chodorow",
    published_date: ISODate("2011-05-06"),
    pages: 68,
    language: "English",
    publisher_id: "oreilly"
}
```

1. Docs.Mongodb.com : https://docs.mongodb.com/
2. Mongodb university : https://university.mongodb.com/

Glace at the MongoDB Query Language (MQL) with TSQL

MongoDB	TSQL
db.inventory.find({})	SELECT * FROM inventory
db.inventory.find({Status:"d"})	SELECT * FROM INVENTORY WHERE STATUS="D"
db.inventory.find({Status:{$in : ["a","d"]}})	SELECT * FROM INVENTORY WHERE STATUS IN ("A","D")
db.inventory.find({Status:"A"{,{qty}:{$lt:30}})	SELECT * FROM INVENTORY WHERE STATUS ="A" AND QTY<30
db.inventory.find({$or:[{Status:"A"},{Qty:{$lt:30}}]})	SELECT * FROM INVENTORY WHERE STATUS="A" OR QTY<30
db.inventory.find({Status:"A" , $OR[{qty:{$lt:30}}, {item: /^p/}])	SELECT * FROM INVENTORY WHERE STATUS="A" OR (QTY<30 AND ITEM LIKE "P%")
Db.inventory.findone()	Select * from inventory limit 1

1

2

1 Docs.Mongodb.com : https://docs.mongodb.com/

2 Mongodb university : https://university.mongodb.com/